Service Deployment Process

This document describes the standard deployment lifecycle from readiness checks through release execution, monitoring, and rollback.

Department: Engineering | Version: 1.1 | Owner: Edward Fuller, Platform Engineering Lead

Access Level: Internal - Engineering | Created: 2024-04-13 | Updated: 2024-06-25 | Source File: engineering_deployment_process_v1.pdf

1.0 Purpose

The deployment process standardizes how engineering teams prepare, execute, validate, and if necessary reverse production releases. A production-level RAG dataset should make it easy to retrieve both the high-level deployment flow and the practical details that matter during a real release window, such as who owns monitoring and when a rollback becomes the safer choice.

The process reduces release risk by requiring readiness evidence, explicit ownership, and post-deployment observation. It recognizes that code quality alone is not enough; the release itself is an operational event that must be planned and managed with the same discipline as the implementation.

2.0 Scope

This process applies to production deployments for customer-facing services, internal platforms that support business-critical operations, and changes involving infrastructure, migrations, or configuration that materially affect runtime behavior.

It covers readiness validation, deployment execution, post-release monitoring, communication, and rollback criteria. Team-specific runbooks may provide tool-level detail, but they should align to this control sequence unless a platform exception is documented.

- Applies to standard releases, scheduled fixes, infrastructure changes, and high-risk migrations.
- Requires a named release owner and a rollback approach before deployment begins.
- Uses release tickets, dashboards, and communication channels as the operational source of truth.

3.0 Responsibilities

The responsibilities below define who must act, approve, review, and maintain records so the process can be executed consistently across teams and audit periods.

3.1 Release Owner

- Confirm readiness evidence, coordinate the release window, and remain available through validation.
- Communicate release status, risks, and rollback decisions in the designated channel.
- Ensure post-release monitoring is active and closure criteria are satisfied before standing down.

3.2 Reviewers and Approvers

- Validate that testing, migration safety, and dependency reviews were completed appropriately.
- Challenge unclear rollout assumptions or missing rollback detail before approval.
- Escalate when the release risk appears inconsistent with the proposed release window or safeguards.

3.3 Platform or On-Call Partners

- Support pipeline execution, environment health review, and incident escalation if needed.
- Help interpret dashboards, alerts, and infrastructure signals during the release.
- Participate in rollback or mitigation when the release impacts shared systems.

4.0 Readiness Validation

Readiness review should confirm more than test success. The release owner must understand schema changes, dependency timing, configuration changes, feature flags, and any business activities that could amplify the blast radius if the rollout goes poorly. Mature teams treat readiness as a decision checkpoint, not a rubber stamp.

Where the release includes data migration, authentication changes, or infrastructure moves, the review should include a clear rollback strategy and a confidence statement about whether rollback is technically safe after partial execution. That decision should not be improvised during the release window.

- Verify code review completion, required tests, and linked release ticket scope.
- Confirm migration sequencing, configuration changes, and dependency coordination.
- Review the rollback plan and identify any steps that become irreversible after execution.

5.0 Deployment Execution

The release should be performed from the approved artifact or branch through the standard deployment workflow. Each stage of the rollout should be observed rather than treated as complete simply because the automation advanced. Automated success is useful, but production behavior is the actual release outcome that matters.

Where staged promotion is available, the service should be validated in lower environments or limited exposure modes before broad rollout. A healthy release is one in which evidence accumulates progressively and the team stays able to pause without confusion or blame if the signals worsen.

- Start the deployment through the approved pipeline and record the start time and operator.
- Validate staging or progressive rollout behavior before expanding exposure.
- Pause immediately if error rates, latency, queue depth, or auth failures move outside acceptable thresholds.

6.0 Post-Deployment Monitoring and Rollback

The release is not complete at the moment the pipeline finishes. The release owner remains responsible for watching health checks, logs, traces, and user-visible flows until the service is stable and any known transient issues are understood. Teams should explicitly decide when the monitoring window ends rather than drifting away from the release without closure.

Rollback should be treated as a disciplined mitigation, not an admission of failure. If the new version introduces user harm, data integrity risk, or ongoing instability, rolling back quickly is often the lowest-cost decision. Waiting for certainty can turn a contained release issue into a broader incident.

- Check core user flows, health endpoints, dashboards, and alerts within the first fifteen minutes.
- Communicate final release state or rollback status in the engineering release channel.
- Capture timeline, observed issues, and follow-up tasks in the release record after stabilization.

7.0 Exceptions and Escalation

Any release that cannot meet the standard readiness controls must document the exception, the compensating control, and the approver. Emergency releases may move faster, but they still require explicit ownership, monitoring, and post-release documentation.

Escalate immediately if rollback is unsafe, if migration behavior appears inconsistent, or if shared infrastructure signals indicate the release is affecting more than the intended service. Early escalation is a control, not a failure.

8.0 Records and Review

Release tickets, dashboard links, approver notes, rollback decisions, and final release summaries should be retained with the deployment record. A release without durable operational notes becomes difficult to audit or learn from later.

The deployment process is reviewed quarterly using incident learnings, rollback frequency, release delay causes, and platform feedback. Repeated failure patterns should lead to stronger automation or better pre-release controls rather than repeated heroics during rollout.